Manual do aluno: Encontro 2, contêineres

Ao final deste encontro, a API da sua dupla roda dentro de um contêiner que vocês mesmos construíram, e você sabe explicar a diferença entre contêiner e máquina virtual. Siga os passos na ordem. Os trechos marcados como **Checkpoint** dizem como saber se você está no caminho certo. Vários passos pedem para você **prever** o resultado antes de executar: escreva a previsão no caderno e compare depois.

Se tiver dificuldade em qualquer passo, peça ajuda à sua IA favorita, como o Claude, o ChatGPT ou o Gemini. Diga em qual passo você está, cole a mensagem de erro inteira e conte o que você esperava ver. Peça que ela explique o motivo, e não só o comando. Antes de rodar qualquer coisa no seu repositório, leia a resposta e confira se faz sentido. Depois anote no caderno o que você aprendeu com a dúvida: o questionário pergunta o que você entendeu. A IA ajuda a destravar, mas o relatório e as respostas do questionário precisam ser suas.

Antes de começar

Faça isto antes de qualquer passo:

1. Abra o **seu** Codespace no repositório da dupla, como no encontro 1. Não crie um novo: no GitHub, clique em **Code** (Código), abra a aba **Codespaces** (Espaços de código) e clique no que já existe.
2. Mantenha os mesmos papéis do encontro 1, o **aluno A** (dono do repositório) e o **aluno B** (colaborador). Cada um trabalha no seu próprio Codespace e faz commits com a própria conta.
3. Tenha à mão a **variação da dupla**, a mesma do encontro 1. Neste encontro você passa a usar também a `PORTA`.
4. Confirme que o Docker funciona. No terminal do Codespace, rode:

```
docker --version
docker version
```

Checkpoint: o segundo comando mostra as seções `Client` e `Server`, sem mensagem de erro. Se aparecer erro, avise o professor antes de seguir, porque o resto do encontro depende disso.

Valor	O que é	Exemplo
`NOME_DUPLA`	Nome único da dupla	`dupla-07`
`MENSAGEM`	Frase que a sua API devolve em `/aluno`	`Olá da dupla 07`
`PORTA`	Número da porta da sua API	`5007`

Nos comandos deste manual, onde estiver escrito `PORTA`, troque pelo seu número. Se o seu valor for `5007`, o comando `curl http://localhost:PORTA/saude` vira `curl http://localhost:5007/saude`.

Papéis neste encontro.

Papel	O que faz
Aluno A	Cria a API Flask (passo 2), roda sem contêiner (passo 3) e envia o código (passo 4). Muda a mensagem e reconstrói a imagem (passo 7)
Aluno B	Baixa o código e repete o passo 3 **do zero**, anotando tudo (passo 4). Escreve o Dockerfile (passo 5) e envia o Dockerfile ao repositório (passo 9)
Os dois	Atualizam o repositório (passo 1), constroem e rodam o contêiner (passo 6) e medem o contêiner (passo 8), cada um no seu Codespace

1. a API sem contêiner

Situação: "Funciona na minha máquina." Você vai criar uma API pequena, rodá-la do jeito tradicional e ver quanto trabalho dá para o seu colega repetir o mesmo.

Passo 1. Atualizar o repositório (os dois). Abra o seu Codespace e, no terminal, rode:

```
git pull
```

Passo 2. Criar a API (aluno A). No painel **Explorer** (lista de arquivos, à esquerda), clique no ícone de **novo arquivo**, digite `app/requirements.txt` e pressione Enter. O editor cria a pasta `app` junto com o arquivo. Repita para `app/app.py` e `app/test_app.py`. No `app.py`, troque os três valores do bloco de variação pelos da sua dupla.

`app/requirements.txt`:

```
flask==3.1.*
pytest>=8
```

`app/app.py`:

```
import os

from flask import Flask, jsonify

# ===== VARIAÇÃO DA DUPLA (mude estes valores) =====
NOME_DUPLA = "dupla-00"
MENSAGEM = "Olá da minha API"
PORTA = 5000
# ================================================

VERSAO = os.environ.get("APP_VERSION", "v1")

app = Flask(__name__)


@app.route("/saude")
def saude():
    return jsonify(status="ok", versao=VERSAO)


@app.route("/aluno")
def aluno():
    return jsonify(dupla=NOME_DUPLA, mensagem=MENSAGEM)


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=PORTA)
```

`app/test_app.py`:

```
from app import app, MENSAGEM, NOME_DUPLA


def test_saude_responde_ok():
    resposta = app.test_client().get("/saude")
    assert resposta.status_code == 200
    assert resposta.get_json()["status"] == "ok"
```

```
def test_aluno_mostra_variacao_da_dupla():
    resposta = app.test_client().get("/aluno")
    assert resposta.status_code == 200
    dados = resposta.get_json()
    assert dados["dupla"] == NOME_DUPLA
    assert dados["mensagem"] == MENSAGEM
```

Na **raiz** do repositório (fora da pasta `app`), crie também o arquivo `.gitignore`. Ele diz ao Git quais pastas não enviar:

```
.venv/
__pycache__/
.pytest_cache/
```

Passo 3. Rodar sem contêiner (aluno A). Preveja: o que as rotas `/saude` e `/aluno` vão devolver? Escreva a previsão no caderno e rode:

```
cd app
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python app.py
```

Se o comando `venv` não existir, rode `sudo apt install python3-venv` e tente de novo. A API fica ocupando este terminal. Abra um **segundo terminal** (ícone **+** no painel do terminal) e teste as duas rotas, trocando `PORTA` pelo seu número:

```
curl http://localhost:PORTA/saude
curl http://localhost:PORTA/aluno
```

Se preferir ver no navegador, abra a aba **PORTS** (Portas), clique no ícone de navegador ao lado da sua porta e acrescente `/saude` ou `/aluno` ao endereço. Depois, volte ao primeiro terminal, pare a API com Ctrl+C e rode os testes:

```
python -m pytest -q
```

Checkpoint: `/aluno` mostra o nome e a mensagem da sua dupla, e os testes terminam com `2 passed`.

Passo 4. Enviar e repetir no colega. O aluno A volta para a raiz do repositório e envia o código:

```
cd ..
git add app .gitignore
git commit -m "API Flask da dupla"
git push
```

O aluno B abre o **seu** Codespace, roda `git pull` e repete o passo 3 **do zero**, no próprio Codespace. Anote em um bloco de notas **todos os comandos que precisou rodar e o que deu errado**. Esse trabalho do aluno B é exatamente o problema que o contêiner resolve, e o questionário vai perguntar sobre ele.

2. a API dentro de um contêiner

Passo 5. Escrever o Dockerfile (aluno B). Na **raiz** do repositório, ao lado da pasta `app`, crie o arquivo `Dockerfile` (sem extensão). Troque o `5000` da linha `EXPOSE` pela sua `PORTA`.

```
FROM python:3.12-slim
WORKDIR /app
COPY app/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ .
EXPOSE 5000
CMD ["python", "app.py"]
```

O que cada linha faz, em uma frase:

- `FROM` escolhe a imagem de partida, com o Python já instalado.
- `WORKDIR` define a pasta de trabalho dentro do contêiner.
- `COPY` e `RUN` (o primeiro par) copiam a lista de dependências e instalam.
- `COPY app/ .` copia o código da API.
- `EXPOSE` documenta a porta da API. Ela não abre a porta sozinha.
- `CMD` diz o comando que o contêiner executa ao iniciar.

Crie também o arquivo `.dockerignore`, na mesma pasta do Dockerfile, para não copiar lixo para a imagem:

```
**/.venv
**/__pycache__
**/.pytest_cache
.git
.github
site
```

O aluno B envia os dois arquivos para o repositório agora, para o aluno A poder usar o mesmo Dockerfile:

```
git add Dockerfile .dockerignore
git commit -m "Dockerfile da API"
git push
```

O aluno A roda `git pull` no seu Codespace e confere que o `Dockerfile` apareceu na lista de arquivos.

Passo 6. Construir e rodar (os dois). Preveja: se eu subir o contêiner **sem** mapear a porta, consigo abrir `localhost:PORTA/saude`? Escreva a previsão. Na **raiz** do repositório, rode:

```
docker build -t projeto-devops:v1 .
docker run -d --name api projeto-devops:v1
curl http://localhost:PORTA/saude
```

Compare com a sua previsão. Agora refaça mapeando a porta (troque `PORTA` pelo seu número **nos dois lugares** do `-p`):

```
docker rm -f api
docker run -d --name api -p PORTA:PORTA projeto-devops:v1
curl http://localhost:PORTA/saude
docker ps
docker logs api
```

Checkpoint: o `curl` devolve `{"status":"ok","versao":"v1"}` e o `docker ps` lista o contêiner `api`.

Passo 7. Cache de camadas (os dois, sem enviar). Preveja: se eu mudar só a `MENSAGEM` no `app.py` e reconstruir a imagem, quais passos do build serão refeitos? Escreva a previsão, mude a `MENSAGEM` em `app/app.py`, salve e rode:

```
docker build -t projeto-devops:v2 .
```

Observe no resultado quais linhas aparecem como `CACHED`. Depois troque o contêiner pela versão nova e confira `/aluno`:

```
docker rm -f api
docker run -d --name api -p PORTA:PORTA projeto-devops:v2
curl http://localhost:PORTA/aluno
```

Essa mudança é só uma experiência: **não faça commit dela**. Para voltar o arquivo ao que está no repositório, rode `git checkout app/app.py`.

Passo 8. Contêiner x máquina virtual (os dois). Meça duas coisas do seu contêiner e complete o quadro no caderno:

```
docker images projeto-devops
time docker run --rm projeto-devops:v1 python -c "print('oi')"
```

Característica	Máquina virtual	Contêiner (o seu)
Tem um sistema operacional completo próprio?		
Tamanho (ordem de grandeza)		valor da coluna SIZE
Tempo para iniciar		valor de `real`
O que compartilha com a máquina hospedeira?		

Passo 9. Conferir e reunir as evidências (os dois). Confirme no GitHub que o `Dockerfile` e o `.dockerignore` estão no repositório, e que o histórico mostra commits dos dois alunos:

```
git pull
git log --oneline -5
```

Suba de novo o contêiner da versão `v1` mapeando a porta e tire dois prints: o `docker ps` com o contêiner rodando e o `curl` com a resposta. Anote o nome e a tag da imagem, a porta mapeada e o número da linha do Dockerfile onde aparece a sua porta.

Antes de sair. Os arquivos enviados com `git push` já estão no GitHub. Imagens e contêineres ficam só no Codespace e não precisam ser guardados. Se quiser parar o contêiner, rode `docker rm -f api`.

Erros comuns

Sintoma	Causa provável	O que fazer
`permission denied` ao falar com o Docker	Seu usuário não pode usar o Docker	Avise o professor; não use `sudo` sem combinar
`port is already allocated`	Outro contêiner usa a porta	Rode `docker ps` e remova o antigo com `docker rm -f NOME`
`The container name "/api" is already in use`	Já existe um contêiner chamado `api`	Rode `docker rm -f api` e repita o `docker run`
`curl` falha com conexão recusada ou resetada	Faltou o `-p`, ou a porta do mapeamento difere da `PORTA` do `app.py`	Compare os três números: `app.py`, `-p` e o `curl`
`ModuleNotFoundError: flask`	O ambiente virtual não está ativo	Rode `source .venv/bin/activate` dentro da pasta `app`
O build não acha `app/requirements.txt`	O `docker build` rodou dentro da pasta `app`	Volte para a raiz do repositório e rode de novo
`toomanyrequests` ao baixar a imagem	O Docker Hub limita os downloads feitos por um mesmo endereço de rede	Espere alguns minutos, tente de novo e avise o professor
`git pull` reclama de arquivos alterados	Você mudou o `app.py` no passo 7 e não desfez	Rode `git checkout app/app.py` e tente de novo
O Codespace parou sozinho	Ficou cerca de 30 minutos sem uso	Reabra em **Code** > **Codespaces** e rode `git pull`; se o Docker não responder, confira com docker version

Relatório da dupla

Depois do passo 9, e antes do questionário, cada aluno escreve o que fez e o que aprendeu, e a dupla junta tudo em um relatório. Isso fixa o encontro e deixa claro quem fez o quê.

Parte 1: cada aluno escreve a sua parte

Aluno A e aluno B respondem **separadamente**, com as próprias palavras, em 3 a 6 linhas por pergunta. Não vale copiar do colega.

1. **O que eu fiz hoje.** Cite os arquivos que criei ou alterei, os comandos que rodei e o meu papel (aluno A ou B).
2. **O que eu aprendi.** Explique com as suas palavras o que é um Dockerfile, uma imagem e um contêiner.
3. **Do Dockerfile ao contêiner.** Descreva, na ordem, o que acontece desde o arquivo `Dockerfile` até a API responder no `curl`.
4. **Contêiner ou máquina virtual.** Qual é a diferença? Cite pelo menos um número que você mediu no passo 8.
5. **O que deu errado ou foi difícil**, e como a dupla resolveu.
6. **Uma dúvida** que ainda tenho.

Mais uma pergunta, que depende do seu papel:

Papel	Pergunta extra
Aluno A	No `Dockerfile`, por que a lista de dependências é copiada e instalada **antes** de copiar o código da API? O que isso mudou no build do passo 7?
Aluno B	Repetir o passo 3 do zero deu trabalho. Liste o que você precisou fazer e o que deu errado, e diga o que o contêiner eliminaria dessa lista.

Parte 2: relatório da dupla

Um único relatório por dupla, escrito em conjunto, com estes itens:

- **Identificação:** o `NOME_DUPLA`, o nome e o usuário do GitHub de cada aluno e o link do repositório.
- **Previsões e resultados** dos passos 3, 6 e 7, no quadro abaixo.
- **Quadro contêiner x máquina virtual** do passo 8, preenchido.
- **Evidências:** os dois prints do passo 9 (`docker ps` e `curl`), o nome e a tag da imagem, a porta mapeada e o número da linha do Dockerfile onde aparece a sua porta.
- **Linha do tempo:** cole o resultado de `git log --oneline -5` e escreva, ao lado de cada commit, quem fez e o que mudou.
- **Um problema e a solução:** algo que travou a dupla e como foi resolvido.
- **O que a dupla aprendeu,** em três frases.

Passo	O que prevemos	O que aconteceu
3. Rotas `/saude` e `/aluno`		

Passo	O que prevemos	O que aconteceu
6. Contêiner sem mapear a porta		
7. Linhas refeitas ao mudar só a `MENSAGEM`		

A entrega é feita pelo Moodle, como explica a seção Entrega, no fim deste manual.

Questionário

As perguntas dependem do repositório e do contêiner da sua dupla: a sua `PORTA`, o nome da imagem, os valores medidos. Por isso, **na hora de responder, deixe abertos o repositório no GitHub e o terminal do Codespace**, ou vá anotando em um bloco de notas o que aconteceu, como pedem os passos 3 a 9.

Antes de abrir o questionário, confira:

- ☐ O repositório da dupla está aberto em uma aba do navegador.
- ☐ O Codespace está aberto, com o terminal pronto para consultar `docker ps`, `docker images` e `git log`.
- ☐ As previsões e os resultados dos passos 3, 6 e 7 estão à mão.
- ☐ O quadro contêiner x máquina virtual do passo 8 está preenchido.
- ☐ A anotação do passo 9 (nome e tag da imagem, porta mapeada, linha do Dockerfile) está à mão.

O questionário mistura três tipos de pergunta: conceitos (contêiner, imagem, Dockerfile, cache), evidências do **seu** contêiner (valores, comandos, histórico) e perguntas abertas, em que você explica com as suas palavras. Uma resposta com o dado certo do seu repositório vale mais do que uma resposta de memória.

Perguntas do questionário

O questionário tem seis blocos. Todos respondem os blocos 1, 2, 3 e 6. O bloco 4 é só do aluno A, e o bloco 5 é só do aluno B.

Bloco 1. Identificação (resposta curta)

- Nome completo.
- O `NOME_DUPLA`.
- O seu usuário no GitHub.
- O seu papel: aluno A ou aluno B.

Bloco 2. Conceitos (múltipla escolha, uma resposta certa)

1. O que um contêiner compartilha com a máquina que o hospeda, e uma máquina virtual não?

- a) Um sistema operacional completo, com o próprio kernel
- b) O kernel do sistema operacional da máquina hospedeira
- c) Um hardware emulado, só dele
- d) Nada: o contêiner é totalmente isolado

2. Qual é a relação entre imagem e contêiner?

- a) A imagem é o modelo pronto, e o contêiner é uma cópia dela em execução
- b) A imagem é o contêiner parado, e não pode gerar outros
- c) O contêiner é o modelo, e a imagem é a cópia em execução
- d) São dois nomes para a mesma coisa

3. O que a linha `EXPOSE` do Dockerfile faz?

- a) Abre a porta da API para a internet
- b) Mapeia a porta do contêiner para a da máquina

- c) Documenta a porta da API, sem abri-la sozinha
- d) Inicia a API

4. No comando `docker run -d --name api -p 5007:5007 projeto-devops:v1`, o que faz o `-p 5007:5007`?

- a) Define a senha do contêiner
- b) Liga a porta 5007 da máquina à porta 5007 do contêiner
- c) Escolhe a versão do Python
- d) Limita o contêiner a 5007 MB de memória

5. Você mudou só a `MENSAGEM` no `app.py` e reconstruiu a imagem. Por que o `pip install` apareceu como `CACHED`?

- a) Porque o Docker não refaz nenhum passo quando o código muda
- b) Porque o `pip install` só roda na primeira vez da vida da máquina
- c) Porque o Docker apagou o cache do `FROM`
- d) Porque o `requirements.txt` não mudou e foi copiado antes do código, então essa camada foi reaproveitada

6. Para que serve o arquivo `.dockerignore`?

- a) Evitar que arquivos desnecessários, como `.venv` e `.git`, sejam copiados para a imagem
- b) Impedir que o contêiner acesse a internet
- c) Esconder o Dockerfile do repositório
- d) Guardar a senha do Docker Hub

Bloco 3. Evidências do seu repositório e do seu contêiner (resposta curta; consulte o repositório, o Codespace ou as suas anotações)

7. Link do repositório da dupla.

8. Os três valores da sua variação: `NOME_DUPLA`, `MENSAGEM` e `PORTA`.

9. Nome e tag da imagem que você construiu (coluna `REPOSITORY` e `TAG` do `docker images`).

10. O comando `docker run` completo que você usou para subir o contêiner com a porta mapeada.

11. Em qual número de linha do `Dockerfile` aparece a sua porta, e qual é o valor escrito nela?

12. Qual é o valor da coluna `SIZE` da sua imagem e o tempo `real` que o `time docker run` mostrou?

13. No build da versão `v2` (passo 7), quais linhas do Dockerfile apareceram como `CACHED`?

14. Rode `git log --format="%h %an %s" -3` e escreva o código curto do commit que adicionou o `Dockerfile`, quem o fez e a mensagem.

Bloco 4. Só para o aluno A (resposta curta)

15. Passo 3: o que você previu que `/saude` e `/aluno` devolveriam, e o que devolveram de verdade? Responda em 2 a 3 linhas.

16. Passo 7: o que você previu que seria refeito no build ao mudar só a `MENSAGEM`, e o que realmente foi refeito? Responda em 2 a 3 linhas.

Bloco 5. Só para o aluno B (resposta curta)

17. Passo 4: liste os comandos que você precisou rodar para fazer a API funcionar do zero no seu Codespace e o que deu errado no caminho. Responda em 3 a 5 linhas.

18. Passo 6: você previu que o `curl` funcionaria sem o `-p`? O que aconteceu, e por que isso acontece? Responda em 2 a 3 linhas.

Bloco 6. Perguntas abertas (todos; responda com as suas palavras)

19. "Funciona na minha máquina." Explique como o contêiner resolve esse problema, usando o que aconteceu com o aluno B no passo 4.

20. Qual é a diferença entre um contêiner e uma máquina virtual? Use pelo menos um número que você mediu no passo 8 para sustentar a resposta.

Entrega

Como fazer o relatório.

1. Cada aluno escreve a **Parte 1**, sozinho, com as próprias palavras.
2. Juntos, vocês montam a **Parte 2**, o relatório da dupla: identificação, quadro de previsões e resultados, quadro contêiner x máquina virtual, evidências (os dois prints, nome e tag da imagem, porta e linha do Dockerfile), linha do tempo dos commits, um problema resolvido e o que a dupla aprendeu.
3. Antes de fechar, confirmem se o link do repositório abre, se o `Dockerfile` e o `.dockerignore` estão lá e se os commits dos dois alunos aparecem na linha do tempo.
4. Juntem tudo em um único arquivo (por exemplo, em PDF), com o `NOME_DUPLA` no nome do arquivo.

Entrega no Moodle. No final da tarefa, a dupla entrega o relatório pelo Moodle. O relatório é o mesmo para os dois, mas **cada aluno entrega o seu, separadamente, pela própria conta**. Confira se o envio foi concluído antes de sair.